TicketHub

Documentação final do MVP – Sistema de gerenciamento de chamados

Aplicativo mobile (React Native) · API REST (Node.js) · PostgreSQL

1 INTRODUÇÃO

1.1 Contexto e problema

Em muitas empresas de pequeno porte, os pedidos de suporte de Tecnologia da Informação chegam por mensagens soltas, sem registro do andamento e sem indicação de quem está atendendo cada solicitação. Isso dificulta o acompanhamento pelo funcionário que pediu ajuda e a organização do trabalho da equipe técnica.

O **TicketHub** é um MVP (Minimum Viable Product) de sistema de gerenciamento de chamados (help desk) desenvolvido como Trabalho de Conclusão de Curso. Ele organiza o suporte em um fluxo simples e rastreável, acessível por um aplicativo mobile: o funcionário abre o chamado, o técnico o assume e, ao concluir o serviço, o encerra.

1.2 Objetivo do documento

Este documento reúne a documentação final do MVP: requisitos, modelagem (casos de uso e classes), arquitetura, banco de dados, API, interface do aplicativo, testes, segurança e adequação à LGPD, além das limitações conhecidas e dos próximos passos. A seção de interface apresenta cada tela do aplicativo, separada por perfil (funcionário e técnico), em mockups de celular.

1.3 Escopo do MVP

Dentro do escopo:

- Autenticação por e-mail e senha, com token JWT.
- Dois perfis de usuário com permissões distintas: cliente (funcionário) e técnico.
- Abertura de chamados com título, descrição e categoria de serviço.
- Listagem de chamados filtrada por perfil, com status e data da última atualização.
- Visualização dos detalhes do chamado, incluindo o técnico responsável.

- Atribuição (assumir) e encerramento de chamados por técnicos.
- Aplicativo mobile para Android e iOS (via Expo).

Fora do escopo, por decisão consciente do MVP:

- Cadastro público de usuários: as contas são criadas internamente pela equipe de T.I.
- Comentários e histórico de interações dentro do chamado.
- Anexos, notificações push e e-mail.
- Prioridade, SLA e relatórios.
- Reabertura de chamados fechados e devolução de chamados à fila.
- Painel administrativo e gestão de usuários e categorias pela interface.

1.4 Perfis de usuário

Na especificação (Etapa 3) o perfil é chamado de **Cliente**: o funcionário da empresa que precisa de suporte. No banco de dados o valor é `CLIENTE` e, na interface do aplicativo, o rótulo exibido é **FUNCIONÁRIO**. Neste documento os termos são equivalentes.

Tabela 1 – Perfis de usuário

Perfil	Valor no banco	O que pode fazer
Cliente (Funcionário)	`CLIENTE`	Abrir chamados; ver apenas os seus chamados; ver os detalhes dos próprios chamados.
Técnico	`TECNICO`	Ver a fila de chamados não encerrados; ver detalhes de qualquer chamado; assumir chamados abertos; encerrar os chamados que assumiu.

Fonte: O autor (2026).

Tabela 2 – Matriz de permissões

Ação	Cliente (Funcionário)	Técnico
Autenticar-se e sair	Sim	Sim
Listar chamados	Sim (somente os seus)	Sim (fila de não encerrados)
Ver detalhes de um chamado	Sim (somente os seus)	Sim (qualquer um)
Abrir chamado	Sim	Não
Assumir chamado	Não	Sim
Encerrar chamado	Não	Sim (somente o responsável)
Listar categorias	Sim	Sim

Fonte: O autor (2026).

1.5 Ciclo de vida de um chamado

Todo chamado percorre três estados, sempre em sequência. Não há transições de volta: um chamado fechado não pode ser reaberto e um chamado em atendimento não retorna à fila.

Figura 1 – Ciclo de vida de um chamado



Fonte: O autor (2026).

Tabela 3 – Status do chamado

Status	Significado	Técnico responsável	Cor no app
`ABERTO`	Aguardando atendimento	Nenhum	Âmbar
`EM_ATENDIMENTO`	Um técnico assumiu o chamado	Definido	Verde
`FECHADO`	Atendimento concluído	Mantido (histórico)	Cinza

Fonte: O autor (2026).

2 TECNOLOGIAS UTILIZADAS

O sistema é composto por três partes: o aplicativo mobile, a API REST e o banco de dados relacional. A comunicação entre o aplicativo e a API ocorre exclusivamente por HTTP/JSON; somente a API acessa o banco.

Tabela 4 – Stack tecnológica

Camada	Tecnologia	Uso
Aplicativo mobile	React Native 0.74 · Expo SDK 51 · React 18	Interface e navegação (React Navigation, pilha nativa)
Persistência local	AsyncStorage	Guarda o token e os dados do usuário no dispositivo
API	Node.js 18+ · Express 4	Rotas, middlewares, controllers e services
Autenticação	`jsonwebtoken` · `bcryptjs`	Emissão de JWT e hash de senhas
Acesso a dados	`pg` (SQL puro)	Consultas parametrizadas, sem ORM
Banco de dados	PostgreSQL 13+	Tabelas, constraints, índices e trigger
Testes	Jest 29	Testes unitários dos services

Fonte: O autor (2026).

3 REQUISITOS

Os requisitos funcionais (RF) seguem a numeração da Etapa 3 do TCC. As regras de negócio (RN) foram consolidadas a partir da implementação, e cada uma indica onde é aplicada.

3.1 Requisitos funcionais

Tabela 5 – Requisitos funcionais e rastreabilidade

ID	Requisito	Casos de uso	Critério de aceite
RF01	Autenticação de usuários	UC01, UC02	Com e-mail e senha válidos de conta ativa, o usuário recebe o JWT e entra; com dados inválidos, recebe “Credenciais inválidas.”
RF02	Controle de acesso por perfil	UC03, UC05, UC06, UC07	Rotas protegidas respondem 401 sem token válido e 403 quando o perfil não tem permissão.
RF03	Gerenciamento de chamados	UC03, UC05, UC06, UC07	Chamados podem ser criados, listados, assumidos e encerrados conforme o perfil.
RF04	Visualização de chamados	UC03	Cliente vê apenas os seus; técnico vê os não encerrados.
RF05	Criação de chamado	UC05	Criado somente com título, descrição e categoria ativa; nasce `ABERTO` e sem técnico.
RF06	Visualização da última atualização	UC03, UC04	Lista e detalhes exibem a data de atualização.
RF07	Visualização do status	UC03, UC04	Status exibido com rótulo e cor.
RF08	Visualização dos detalhes do chamado	UC04	Detalhes mostram número, título, status, descrição, categoria, datas e técnico (ou “Ainda não atribuído”).
RF09	Atribuição de técnico	UC06	Somente técnicos assumem; somente chamados `ABERTO`; status passa a `EM_ATENDIMENTO`.
RF10	Encerramento de chamado	UC07	Status passa a `FECHADO`; chamado já fechado não é encerrado de novo.
RF11	Restrição de encerramento	UC07	Somente o técnico responsável encerra o chamado.

Fonte: O autor (2026).

3.2 Regras de negócio

Tabela 6 – Regras de negócio

ID	Regra	Aplicada em
RN01	Não existe cadastro público; as contas são criadas pelo T.I.	Ausência de rota/tela de registro

ID	Regra	Aplicada em
RN02	Somente usuários ativos (`ativo = TRUE`) conseguem autenticar-se.	`usuarioRepository.buscarPorEmail`
RN03	A falha de login usa mensagem única (“Credenciais inválidas.”), sem revelar se o e-mail existe.	`authService.login`
RN04	Somente clientes abrem chamados.	`authorize('CLIENTE')` e `chamadoService.criar`
RN05	Título e descrição são obrigatórios e não podem conter apenas espaços.	`chamadoService.criar`
RN06	A categoria deve existir e estar ativa.	`chamadoService.criar`
RN07	Todo chamado nasce `ABERTO` e sem técnico.	Default do banco e `chamadoRepository.criar`
RN08	O cliente só visualiza chamados que ele mesmo abriu.	`chamadoService.listar` e `buscarDetalhes`
RN09	O técnico visualiza os detalhes de qualquer chamado.	`chamadoService.buscarDetalhes`
RN10	Somente técnicos assumem chamados.	`authorize('TECNICO')` e `chamadoService.assumir`
RN11	Só é possível assumir chamados com status `ABERTO` (409 caso contrário).	`chamadoService.assumir`
RN12	Um chamado tem no máximo um técnico responsável.	Coluna `tecnico_id`
RN13	Somente o técnico responsável encerra o chamado.	`chamadoService.encerrar`
RN14	Chamado já `FECHADO` não pode ser encerrado novamente (409).	`chamadoService.encerrar`
RN15	A data de atualização é renovada automaticamente a cada alteração.	Trigger `trg_chamado_atualizado_em`
RN16	Categorias inativas somem das listagens, mas o histórico é preservado.	`categoriaRepository.listarAtivas`
RN17	O hash da senha nunca é retornado pela API.	`authService.login`

Fonte: O autor (2026).

3.3 Requisitos não funcionais

A tabela abaixo reúne os requisitos não funcionais verificáveis na implementação. Os identificadores RNF01 (controle de acesso por perfil) e RNF06 (credenciais fornecidas pela empresa, sem autocadastro) são os citados nos casos de uso da Etapa 3.

Tabela 7 – Requisitos não funcionais observados

Categoria	Requisito	Evidência
Controle de acesso (RNF01)	Acesso restrito por perfil; cliente isolado dos dados de outros clientes.	`authorize.js`, `chamadoService.js`
Autenticação (RNF06)	Credenciais fornecidas pela empresa; sem autocadastro.	Ausência de rota de registro
Segurança	Senhas armazenadas somente como hash bcrypt.	`authService.js`, `schema.sql`
Segurança	Sessão por JWT com expiração configurável (padrão 8 h).	`authService.js`, `.env.example`
Segurança	Consultas SQL parametrizadas (mitiga SQL Injection).	`repositories/*.js`
Portabilidade	Aplicativo multiplataforma (Android e iOS).	Expo / React Native
Manutenibilidade	Camadas separadas: rota, controller, service e repository.	`backend/src/`
Testabilidade	Regras de negócio testáveis sem banco, por injeção de dependência.	`services/*.js`, `tests/`
Usabilidade	Interface em português, com status identificado por cor e rótulo.	`theme/colors.js`, telas
Disponibilidade	Endpoint de verificação de saúde da API.	`GET /health`

Fonte: O autor (2026).

4 MODELAGEM DO SISTEMA

4.1 Atores

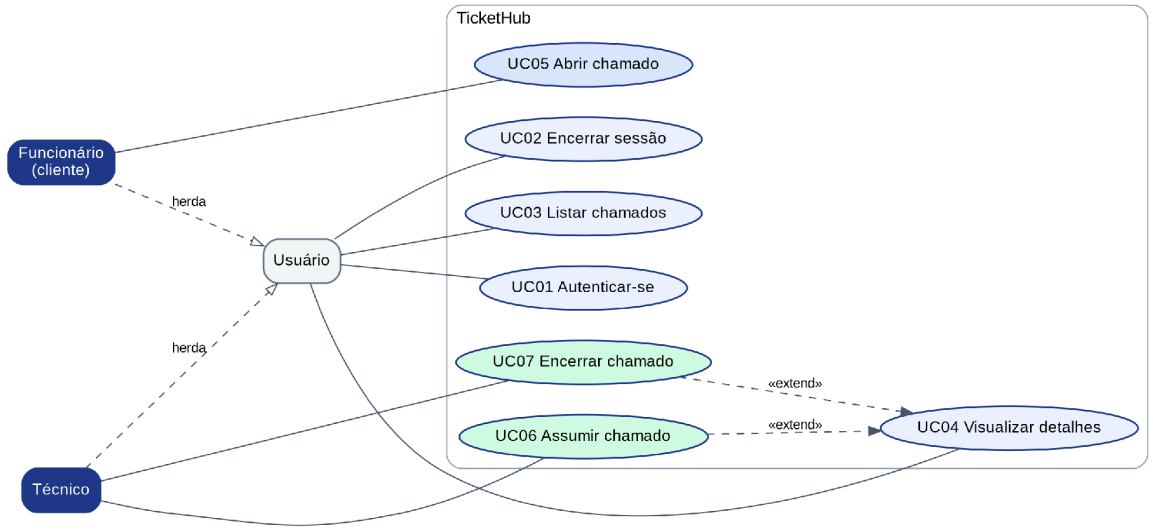
Tabela 8 – Atores do sistema

Ator	Descrição
Usuário	Ator abstrato que representa qualquer pessoa com conta no sistema. Concentra as funcionalidades comuns aos dois perfis (autenticar-se, listar chamados, ver detalhes e sair). Cliente e Técnico herdam suas permissões.
Cliente	Funcionário da empresa que precisa de suporte. Abre chamados e acompanha apenas os que ele mesmo abriu.
Técnico	Profissional de suporte responsável pelo atendimento. Visualiza a fila de chamados, assume chamados e encerra aqueles pelos quais é responsável.

Fonte: O autor (2026).

4.2 Diagrama de casos de uso

Figura 2 – Diagrama de casos de uso do TicketHub



Fonte: O autor (2026).

Tabela 9 – Resumo dos casos de uso

UC	Nome	Ator	Endpoint	Tela
UC01	Autenticar-se	Usuário	`POST /api/auth/login`	Login
UC02	Encerrar sessão	Usuário	— (local)	Lista (botão de sair)
UC03	Listar chamados	Usuário	`GET /api/chamados`	Lista de chamados
UC04	Visualizar detalhes do chamado	Usuário	`GET /api/chamados/:id`	Detalhes do chamado

UC	Nome	Ator	Endpoint	Tela
UC05	Abrir chamado	Cliente	`GET /api/categorias` e `POST /api/chamados`	Novo chamado
UC06	Assumir chamado	Técnico	`PATCH /api/chamados/:id/assumir`	Detalhes do chamado
UC07	Encerrar chamado	Técnico	`PATCH /api/chamados/:id/encerrar`	Detalhes do chamado

Fonte: O autor (2026).

4.3 Descrição dos casos de uso

Tabela 10 – UC01 – Autenticar-se

Ator	Usuário (Cliente ou Técnico)
Objetivo	Permitir que o usuário acesse o sistema com as credenciais fornecidas pela empresa.
Pré-condições	O usuário possui conta ativa, criada previamente pela equipe de T.I. O aplicativo está aberto e não há sessão ativa.
Pós-condições	O usuário está autenticado, o JWT foi emitido e guardado no dispositivo, e a lista de chamados é exibida.
Fluxo principal	1. O usuário informa e-mail e senha na tela de login. 2. O usuário aciona o botão “Entrar”. 3. O sistema verifica se os dois campos foram preenchidos. 4. O sistema busca um usuário ativo com o e-mail informado. 5. O sistema compara a senha informada com o hash armazenado (bcrypt). 6. O sistema gera o JWT (contendo id e perfil) e devolve os dados do usuário. 7. O aplicativo guarda a sessão e exibe a lista de chamados.
Fluxos alternativos	A1 – Campos vazios: o sistema informa “Informe e-mail e senha.” e permanece na tela de login. A2 – E-mail inexistente, conta inativa ou senha incorreta: “Credenciais inválidas.” (mensagem única, para não revelar qual dado está errado). A3 – Falha de comunicação com o servidor: o aplicativo exibe mensagem de erro e o usuário pode tentar novamente.
Regras e requisitos	RF01, RNF01, RNF06. Não existe autocadastro.

Fonte: O autor (2026).

Tabela 11 – UC02 – Encerrar sessão

Ator	Usuário (Cliente ou Técnico)
Objetivo	Sair do sistema, removendo a sessão guardada no dispositivo.
Pré-condições	O usuário está autenticado.
Pós-condições	O token e os dados do usuário foram removidos do dispositivo e a tela de login é exibida.

Fluxo principal	1. O usuário aciona a opção de sair na tela de listagem de chamados. 2. O aplicativo remove o token e os dados do usuário do armazenamento local. 3. O aplicativo exibe a tela de login.
Fluxos alternativos	Não há. Observação: o logout é somente local; o JWT já emitido continua válido até expirar.
Regras e requisitos	RF01, RNF01.

Fonte: O autor (2026).

Tabela 12 – UC03 – Listar chamados

Ator	Usuário (Cliente ou Técnico)
Objetivo	Exibir os chamados que o perfil do usuário tem permissão de acompanhar.
Pré-condições	O usuário está autenticado.
Pós-condições	A lista é exibida com data da última atualização, título e status de cada chamado.
Fluxo principal	1. O usuário acessa a tela de listagem (ou atualiza puxando a tela para baixo). 2. O aplicativo solicita a lista ao servidor, enviando o token. 3. O sistema identifica o perfil do usuário. 4. Cliente: retorna somente os chamados abertos por ele, do mais recentemente atualizado para o mais antigo. 5. Técnico: retorna os chamados não encerrados (abertos e em atendimento), do mais antigo para o mais novo. 6. O aplicativo exibe a lista com data de atualização, título e status colorido.
Fluxos alternativos	A1 – Nenhum chamado: “Nenhum chamado encontrado.” A2 – Token ausente, inválido ou expirado: o servidor responde 401 e a lista não é retornada.
Regras e requisitos	RF03, RF04, RF06, RF07. O cliente nunca visualiza chamados de outros clientes (RNF01).

Fonte: O autor (2026).

Tabela 13 – UC04 – Visualizar detalhes do chamado

Ator	Usuário (Cliente ou Técnico)
Objetivo	Consultar todas as informações de um chamado específico.
Pré-condições	O usuário está autenticado. O chamado selecionado existe.
Pós-condições	Os detalhes são exibidos e as ações permitidas ao perfil ficam disponíveis (UC06 e UC07, no caso do técnico).

Fluxo principal	1. O usuário seleciona um chamado na lista (UC03). 2. O aplicativo solicita os dados do chamado ao servidor. 3. O sistema localiza o chamado. 4. O sistema verifica a permissão: o cliente só acessa chamados que abriu; o técnico acessa qualquer chamado. 5. O aplicativo exibe número, título, status, descrição, categoria, datas de criação e atualização e o técnico responsável (ou “Ainda não atribuído”).
Fluxos alternativos	A1 – Chamado inexistente: “Chamado não encontrado.” (404). A2 – Cliente tenta acessar chamado de outro cliente: “Você não tem permissão para ver este chamado.” (403).
Regras e requisitos	RF08, RF06, RF07, RNF01.

Fonte: O autor (2026).

Tabela 14 – UC05 – Abrir chamado

Ator	Cliente
Objetivo	Registrar uma nova solicitação de suporte.
Pré-condições	O usuário está autenticado com o perfil Cliente. Existem categorias de serviço ativas cadastradas.
Pós-condições	O chamado é criado com status ABERTO, vinculado ao cliente e sem técnico responsável, e passa a aparecer na lista do cliente.
Fluxo principal	1. O cliente aciona “Novo chamado” na tela de listagem. 2. O aplicativo carrega as categorias ativas e exibe o formulário. 3. O cliente informa título, descrição e seleciona a categoria de serviço. 4. O cliente aciona “Criar chamado”. 5. O sistema valida perfil, título, descrição e categoria. 6. O sistema registra o chamado com status ABERTO e cliente igual ao usuário autenticado. 7. O aplicativo retorna à lista, que passa a exibir o novo chamado.
Fluxos alternativos	A1 – Campo obrigatório vazio: “Preencha título, descrição e categoria de atendimento.” (o aplicativo não envia). A2 – Categoria inexistente ou inativa: “Categoria de serviço inválida.” (400). A3 – Perfil Técnico: acesso negado (403) já na rota; o service mantém uma segunda barreira (“Apenas clientes podem abrir chamados.”).
Regras e requisitos	RF05, RF02. Título, descrição e categoria são obrigatórios.

Fonte: O autor (2026).

Tabela 15 – UC06 – Assumir chamado

Ator	Técnico
Objetivo	Tornar-se o responsável pelo atendimento de um chamado ainda não atribuído.

Pré-condições	O usuário está autenticado com o perfil Técnico. O chamado está com status ABERTO. O técnico está na tela de detalhes do chamado (UC04).
Pós-condições	O técnico passa a ser o responsável, o status muda para EM_ATENDIMENTO e a data de atualização é renovada.
Fluxo principal	1. O técnico abre os detalhes de um chamado ABERTO. 2. O aplicativo exibe o botão “Assumir chamado”. 3. O técnico aciona o botão. 4. O sistema verifica o perfil e se o chamado existe e está ABERTO. 5. O sistema registra o técnico como responsável e altera o status para EM_ATENDIMENTO. 6. O aplicativo recarrega os detalhes, agora exibindo o técnico responsável.
Fluxos alternativos	A1 – Chamado já em atendimento ou encerrado: “Este chamado já está em atendimento ou foi encerrado.” (409). A2 – Chamado inexistente: 404. A3 – Perfil Cliente: acesso negado (403).
Regras e requisitos	RF09, RF02. Um chamado possui no máximo um técnico responsável.

Fonte: O autor (2026).

Tabela 16 – UC07 – Encerrar chamado

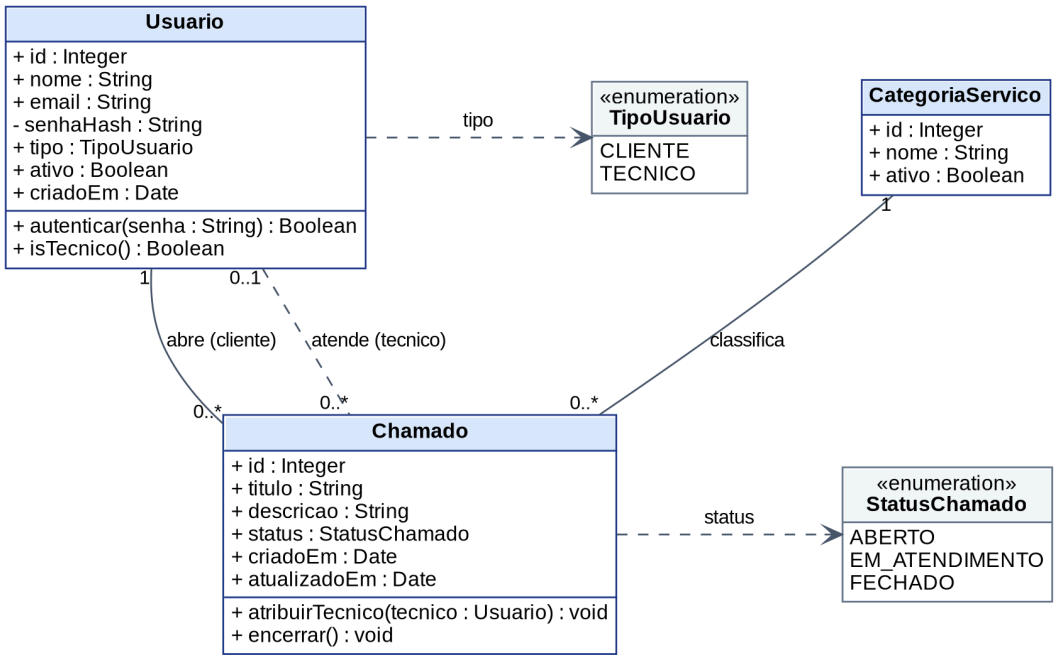
Ator	Técnico (responsável pelo chamado)
Objetivo	Finalizar o atendimento após a conclusão do serviço.
Pré-condições	O usuário está autenticado com o perfil Técnico. O chamado está EM_ATENDIMENTO e atribuído a esse técnico. O técnico está na tela de detalhes do chamado (UC04).
Pós-condições	O status do chamado muda para FECHADO e a data de atualização é renovada.
Fluxo principal	1. O técnico abre os detalhes de um chamado que assumiu. 2. O aplicativo exibe o botão “Fechar chamado” (somente ao técnico responsável, com o chamado em atendimento). 3. O técnico aciona o botão. 4. O sistema verifica perfil, existência do chamado, se o técnico é o responsável e se o chamado ainda não está fechado. 5. O sistema altera o status para FECHADO. 6. O aplicativo recarrega os detalhes e exibe o status “Fechado”.
Fluxos alternativos	A1 – Técnico que não é o responsável: “Apenas o técnico responsável pode encerrar este chamado.” (403). A2 – Chamado já encerrado: “Este chamado já está encerrado.” (409). A3 – Perfil Cliente: acesso negado (403).
Regras e requisitos	RF10, RF11. Somente o técnico responsável pode encerrar o chamado.

Fonte: O autor (2026).

4.4 Diagrama de classes

O diagrama de classes representa o modelo de domínio: as entidades manipuladas pelos casos de uso, seus atributos, operações e relacionamentos. As operações (`autenticar`, `atribuirTecnico`, `encerrar`) não existem no banco; são implementadas na camada de services da API.

Figura 3 – Diagrama de classes do TicketHub



Fonte: O autor (2026).

Tabela 17 – Descrição das classes

Classe	Descrição
Usuario	Pessoa com acesso ao sistema, identificada por e-mail e senha (armazenada apenas como hash). O atributo `tipo` define se é Cliente ou Técnico.
CategoriaServico	Categoria de atendimento escolhida na abertura do chamado (ex.: Erro de rede, Hardware, Software). O atributo `ativo` permite retirar uma categoria da lista sem apagar o histórico.
Chamado	Solicitação de suporte, com título, descrição, status e datas de criação e atualização. `atribuirTecnico` define o responsável e muda o status para EM_ATENDIMENTO; `encerrar` muda o status para FECHADO.
TipoUsuario (enum)	Perfis possíveis de um usuário: CLIENTE e TECNICO.
StatusChamado (enum)	Situações possíveis de um chamado: ABERTO, EM_ATENDIMENTO e FECHADO.

Fonte: O autor (2026).

Tabela 18 – Relacionamentos

Relacionamento	Multiplicidade	Papel	Significado
Usuario abre Chamado	1 para 0..*	cliente	Todo chamado é aberto por exatamente um cliente; um cliente pode abrir vários chamados.
Usuario atende Chamado	0..1 para 0..*	tecnico	Um chamado tem no máximo um técnico responsável (nenhum enquanto aberto); um técnico pode atender vários chamados.
CategoriaServico classifica Chamado	1 para 0..*	—	Todo chamado pertence a uma categoria; uma categoria pode classificar vários chamados.
Usuario → TipoUsuario	dependência	tipo	O perfil do usuário assume um valor da enumeração.
Chamado → StatusChamado	dependência	status	A situação do chamado assume um valor da enumeração.

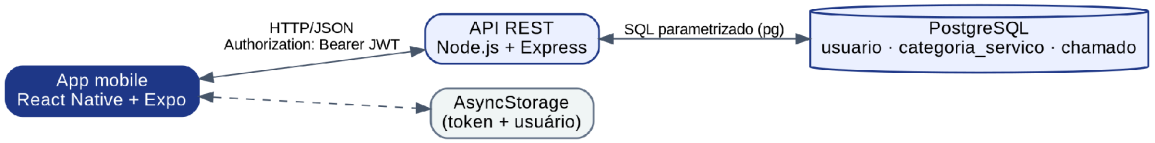
Fonte: O autor (2026).

5 ARQUITETURA

5.1 Visão geral

O aplicativo e a API se comunicam exclusivamente por HTTP/JSON. A autenticação é stateless: cada requisição carrega o JWT no cabeçalho `Authorization: Bearer`, e o servidor não guarda sessão. Apenas a API acessa o banco de dados; o aplicativo nunca fala diretamente com o PostgreSQL.

Figura 4 – Arquitetura geral do TicketHub

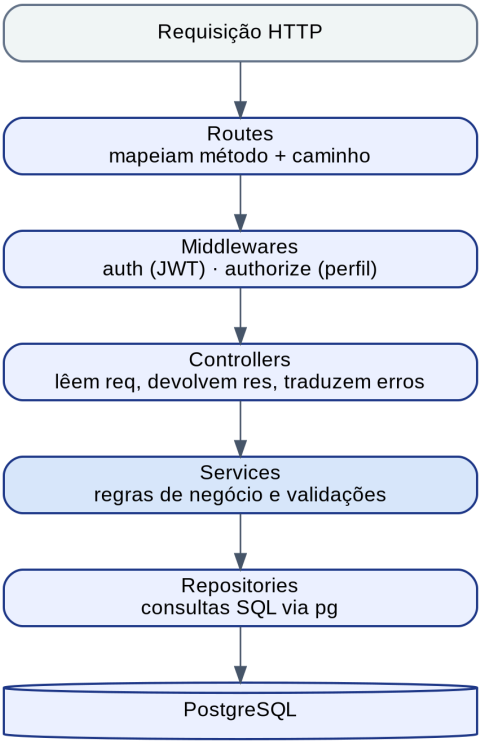


Fonte: O autor (2026).

5.2 Arquitetura do backend em camadas

O backend segue uma arquitetura em camadas, em que cada camada conhece apenas a imediatamente abaixo. Os services nunca importam o Express e os repositories nunca conhecem regras de negócio, o que torna as regras testáveis sem HTTP e sem banco.

Figura 5 – Camadas do backend



Fonte: O autor (2026).

Tabela 19 – Responsabilidade de cada camada

Camada	Pasta	Responsabilidade	HTTP	SQL
Routes	`src/routes/`	Associar rotas a controllers e middlewares	Sim	Não
Middlewares	`src/middlewares/`	Autenticar (JWT) e autorizar (perfil)	Sim	Não
Controllers	`src/controllers/`	Extrair dados da requisição e formatar resposta e erro	Sim	Não
Services	`src/services/`	Regras de negócio e validações	Não	Não
Repositories	`src/repositories/`	Consultas SQL	Não	Sim
Config / Utils	`src/config/`, `src/utils/`	Pool de conexão e `AppError`	Não	—

Fonte: O autor (2026).

5.3 Injeção de dependência

Os services são criados por funções fábrica que recebem suas dependências por parâmetro. A montagem real (service com repository de verdade) acontece em um único lugar, nos arquivos `*Instance.js`. Nos testes, o service é criado manualmente com repositories falsos (mocks do Jest), sem tocar no PostgreSQL.

```
// chamadoService.js
```

```
function createChamadoService({ chamadoRepository, categoriaRepository }) { ... }

// chamadoServiceInstance.js  (produção)
module.exports = createChamadoService({ chamadoRepository, categoriaRepository });
```

5.4 Tratamento de erros

A classe `AppError` estende `Error` com um `statusCode` HTTP. Os services lançam `AppError` para falhas de regra e os controllers respondem com `{ "erro": "<mensagem>" }`. Todas as respostas de erro usam a chave `erro`, em português, o que simplifica o tratamento no aplicativo.

Tabela 20 – Origem dos erros e resposta

Origem	Status	Mensagem
`AppError` **lançado pelo service**	o do erro	Mensagem da regra violada
Middleware `auth`	401	Token não informado. / Token inválido ou expirado.
Middleware `authorize`	403	Você não tem permissão para esta ação.
Rota inexistente	404	Rota não encontrada.
Erro inesperado (ex.: banco)	500	Mensagem interna do erro (ver limitações)

Fonte: O autor (2026).

5.5 Arquitetura do aplicativo

O estado global do aplicativo é mínimo: apenas a sessão (`usuario`), mantida pela Context API (`AuthContext`). O estado de cada tela (listas, formulários, carregamento e erros) fica em `useState` local. A navegação é condicional: o `AppNavigator` monta pilhas diferentes conforme exista ou não um usuário na sessão, de modo que login e logout trocam de tela automaticamente. Todas as chamadas de rede passam por `services/api.js`, que injeta o token e padroniza os erros.

5.6 Decisões de projeto

Tabela 21 – Decisões de projeto e motivos

Decisão	Motivo
Camadas e factory functions	Regra de negócio testável sem banco nem HTTP
`AppError` **com** `statusCode`	Services expressam falhas de negócio sem conhecer HTTP
JWT stateless	Simplicidade; sem tabela de sessões no MVP

Decisão	Motivo
Tabela `usuario` única (coluna `tipo`)	Evita duplicação; os perfis têm os mesmos dados cadastrais
Enums como `VARCHAR` + `CHECK`	Portável e simples; evita `CREATE TYPE` no MVP
SQL puro com `pg` (sem ORM)	Controle total das consultas e menos dependências
`atualizado_em` **por trigger**	Consistência independentemente de quem altera os dados
Sem cadastro público	Requisito de controle de acesso: contas criadas pelo T.I.

Fonte: O autor (2026).

5.7 Estrutura do repositório

```
tickethub/
├── backend/            API REST (Node.js + Express)
│   ├── server.js       Ponto de entrada
│   ├── src/            routes · middlewares · controllers · services
│   │                   repositories · config · utils
│   └── tests/          Testes unitários (Jest)
├── frontend/           App mobile (React Native + Expo)
│   └── src/            screens · navigation · context · services · theme
├── docs/               Documentação técnica em Markdown
├── schema.sql          Criação do banco + dados iniciais
└── README.md
```

6 BANCO DE DADOS

6.1 SGBD e regras de transformação

Foi utilizado o PostgreSQL, sistema gerenciador de banco de dados relacional. O diagrama de classes (seção 4.4) foi transportado para o modelo relacional seguindo as regras abaixo.

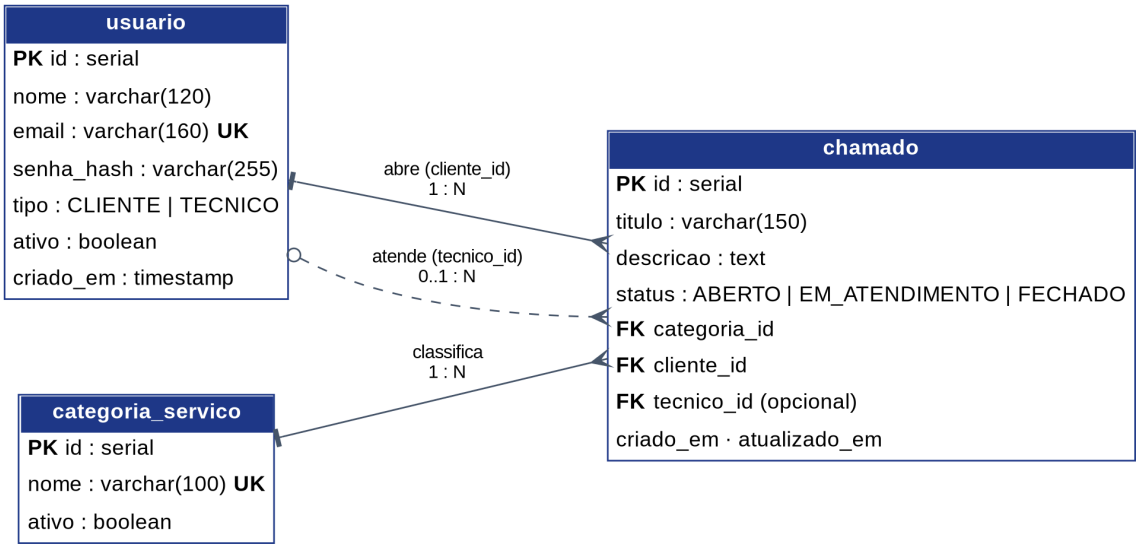
Tabela 22 – Regras de transformação do diagrama de classes

Elemento no diagrama de classes	Elemento no banco de dados
Classe	Tabela com nome em minúsculas e underline (`CategoriaServico` → `categoria_servico`)
Atributo	Coluna com tipo equivalente (Integer → SERIAL/INTEGER; String → VARCHAR/TEXT; Boolean → BOOLEAN; Date → TIMESTAMP)
Atributo `id`	Chave primária com geração automática (`SERIAL`)
Associação 1 para 0..*	Chave estrangeira no lado “muitos”, obrigatória (`NOT NULL`)
Associação 0..1 para 0..*	Chave estrangeira no lado “muitos”, opcional (aceita `NULL`), como em `tecnico_id`
Enumeração	Coluna `VARCHAR` com restrição `CHECK`
Atributo único (email, nome)	Restrição `UNIQUE`
Operações (métodos)	Não geram objetos no banco; são implementadas na camada de services

Fonte: O autor (2026).

6.2 Modelo entidade-relacionamento

Figura 6 – Modelo entidade-relacionamento



Fonte: O autor (2026).

6.3 Dicionário de dados

Tabela 23 – Tabela usuario

Coluna	Tipo	Restrições	Descrição
`id`	SERIAL	PK	Identificador
`nome`	VARCHAR(120)	NOT NULL	Nome completo
`email`	VARCHAR(160)	NOT NULL, UNIQUE	Login do usuário
`senha_hash`	VARCHAR(255)	NOT NULL	Hash bcrypt da senha
`tipo`	VARCHAR(20)	NOT NULL, CHECK (CLIENTE, TECNICO)	Perfil
`ativo`	BOOLEAN	NOT NULL, DEFAULT TRUE	Conta ativa (inativa não faz login)
`criado_em`	TIMESTAMP	NOT NULL, DEFAULT agora	Data de criação

Fonte: O autor (2026).

Tabela 24 – Tabela categoria_servico

Coluna	Tipo	Restrições	Descrição
`id`	SERIAL	PK	Identificador
`nome`	VARCHAR(100)	NOT NULL, UNIQUE	Nome exibido no app
`ativo`	BOOLEAN	NOT NULL, DEFAULT TRUE	Se FALSE, some da lista sem apagar histórico

Fonte: O autor (2026).

Tabela 25 – Tabela chamado

Coluna	Tipo	Restrições	Descrição
`id`	SERIAL	PK	Número do chamado
`titulo`	VARCHAR(150)	NOT NULL	Resumo
`descricao`	TEXT	NOT NULL	Detalhamento do problema
`status`	VARCHAR(20)	NOT NULL, DEFAULT ABERTO, CHECK (ABERTO, EM_ATENDIMENTO, FECHADO)	Situação
`categoria_id`	INTEGER	NOT NULL, FK → categoria_servico(id)	Categoria
`cliente_id`	INTEGER	NOT NULL, FK → usuario(id)	Quem abriu
`tecnico_id`	INTEGER	FK → usuario(id), aceita NULL	Responsável (nulo até ser assumido)
`criado_em`	TIMESTAMP	NOT NULL, DEFAULT agora	Abertura
`atualizado_em`	TIMESTAMP	NOT NULL, atualizado por trigger	Última atualização

Fonte: O autor (2026).

6.4 Índices e trigger

Foram criados índices nas colunas `cliente_id` (lista do cliente), `tecnico_id` (chamados por técnico) e `status` (fila do técnico), as consultas mais frequentes. As colunas únicas (`email` e `nome` da categoria) ganham índice implícito.

A trigger `trg_chamado_atualizado_em` renova a coluna `atualizado_em` a cada `UPDATE` na tabela `chamado` (assumir ou encerrar). Por isso o código da aplicação nunca altera essa coluna (RN15).

```
CREATE OR REPLACE FUNCTION set_atualizado_em() RETURNS TRIGGER AS $$
BEGIN
    NEW.atualizado_em = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_chamado_atualizado_em
    BEFORE UPDATE ON chamado
    FOR EACH ROW EXECUTE FUNCTION set_atualizado_em();
```

> O banco não garante que `cliente_id` seja um usuário `CLIENTE` nem que `tecnico_id` seja um `TECNICO`; essa verificação é feita na camada de services.

6.5 Dados iniciais (seed)

O script `schema.sql` cria as tabelas, as restrições, os índices e a trigger e insere dados de teste: dois usuários (senha `123456`), sete categorias e chamados de exemplo, todos abertos e sem técnico. O script começa com `DROP TABLE IF EXISTS`, portanto deve ser usado somente em desenvolvimento.

Tabela 26 – Usuários de teste

Nome	E-mail	Perfil
Antonio Carlos Silveira	antonio@test.com	CLIENTE (exibido como FUNCIONÁRIO)
Carlos Silva	carlos.silva@test.com	TECNICO

Fonte: O autor (2026).

Categorias: Erro de rede, Recuperacao de dados, Hardware, Software, Acesso e senha, Impressora e Outros. Os textos de categoria do seed não usam acentos de propósito, para evitar problemas de codificação em terminais Windows.

6.6 Consultas usadas pela aplicação

Tabela 27 – Operações e consultas SQL

Operação	Repository	SQL (resumo)
Buscar usuário por e-mail	`usuarioRepository`	`SELECT * FROM usuario WHERE email = $1 AND ativo = TRUE`
Listar categorias ativas	`categoriaRepository`	`SELECT id, nome FROM categoria_servico WHERE ativo = TRUE ORDER BY nome`
Criar chamado	`chamadoRepository`	`INSERT INTO chamado (titulo, descricao, categoria_id, cliente_id) VALUES ($1,$2,$3,$4)`
Listar do cliente	`chamadoRepository`	`... WHERE c.cliente_id = $1 ORDER BY c.atualizado_em DESC`
Listar para técnico	`chamadoRepository`	`... WHERE c.status != 'FECHADO' ORDER BY c.criado_em ASC`
Assumir	`chamadoRepository`	`UPDATE chamado SET tecnico_id = $1, status = 'EM_ATENDIMENTO' WHERE id = $2`
Alterar status	`chamadoRepository`	`UPDATE chamado SET status = $1 WHERE id = $2`

Fonte: O autor (2026).

As consultas de chamado partem de uma base comum que junta `chamado` com `categoria_servico` e com o cliente (JOIN) e com o técnico (LEFT JOIN, pois

pode ser nulo), retornando os campos `categoria_nome`, `cliente_nome` e `tecnico_nome`.

7 API REST

A API trabalha com JSON. Em desenvolvimento, a URL base é `http://localhost:3000`. As rotas protegidas exigem o cabeçalho `Authorization: Bearer <token>`. Campos vindos do banco são retornados em `snake_case` (`criado_em`, `tecnico_nome`) e os de entrada em `camelCase` (`categoriaId`). Os erros seguem o formato `{ "erro": "mensagem" }`.

Tabela 28 – Endpoints

Método	Rota	Perfil	Descrição
GET	`/health`	Público	Verifica se a API está no ar
POST	`/api/auth/login`	Público	Autentica e devolve o JWT
GET	`/api/categorias`	Qualquer	Lista categorias ativas
GET	`/api/chamados`	Qualquer	Lista chamados (filtrada por perfil)
GET	`/api/chamados/:id`	Dono ou técnico	Detalhes do chamado
POST	`/api/chamados`	CLIENTE	Abre um chamado
PATCH	`/api/chamados/:id/assumir`	TECNICO	Assume um chamado ABERTO
PATCH	`/api/chamados/:id/encerrar`	TECNICO responsável	Encerra o chamado

Fonte: O autor (2026).

Tabela 29 – Códigos de status HTTP utilizados

Código	Quando
200	Sucesso em leitura ou atualização
201	Chamado criado
400	Dados inválidos ou ausentes
401	Credenciais inválidas; token ausente, inválido ou expirado
403	Perfil sem permissão ou recurso de outro usuário
404	Chamado ou rota inexistente
409	Conflito de estado (ex.: assumir chamado já em atendimento)
500	Erro interno

Fonte: O autor (2026).

7.1 Login

`POST /api/auth/login` autentica um usuário ativo. O token contém `{ id, tipo }` e expira conforme `JWT_EXPIRES_IN` (padrão 8 h). O hash da senha nunca é retornado.

```
// Requisição
{ "email": "carlos.silva@test.com", "senha": "123456" }

// Resposta 200
{ "token": "eyJhbGciOiJIUzI1NiIs...",
  "usuario": { "id": 2, "nome": "Carlos Silva",
               "email": "carlos.silva@test.com", "tipo": "TECNICO" } }
```

7.2 Objeto Chamado

```
{ "id": 1, "titulo": "Backup nao esta funcionando",
  "descricao": "...", "status": "ABERTO",
  "criado_em": "...", "atualizado_em": "...",
  "categoria_id": 2, "categoria_nome": "Recuperacao de dados",
  "cliente_id": 1, "cliente_nome": "Antonio Carlos Silveira",
  "tecnico_id": null, "tecnico_nome": null }
```

7.3 Regras e erros por endpoint

Tabela 30 – Erros por endpoint

Endpoint	Status	Mensagem	Causa
`POST /auth/login`	400	Informe e-mail e senha.	Campo ausente ou vazio
`POST /auth/login`	401	Credenciais inválidas.	E-mail inexistente, conta inativa ou senha errada
`GET /chamados/:id`	404	Chamado não encontrado.	Não existe
`GET /chamados/:id`	403	Você não tem permissão para ver este chamado.	Cliente tentando ver chamado de outro
`POST /chamados`	403	Você não tem permissão para esta ação.	Usuário é técnico
`POST /chamados`	400	O título é obrigatório. / A descrição é obrigatória.	Campo vazio
`POST /chamados`	400	Categoria de serviço inválida.	Categoria inexistente ou inativa
`PATCH .../assumir`	409	Este chamado já está em atendimento ou foi encerrado.	Status diferente de ABERTO
`PATCH .../encerrar`	403	Apenas o técnico responsável pode encerrar este chamado.	Outro técnico ou chamado sem responsável
`PATCH .../encerrar`	409	Este chamado já está encerrado.	Já FECHADO

Fonte: O autor (2026).

7.4 Cliente HTTP do aplicativo

O aplicativo consome a API pelo módulo `frontend/src/services/api.js`. A função interna `request` lê o token no AsyncStorage, envia a requisição com `Content-Type: application/json` e, se houver token, `Authorization: Bearer`. Em caso de erro, lança uma exceção com a mensagem retornada em `erro`, de modo que as telas só precisam de `try/catch`.

Tabela 31 – Funções do cliente HTTP

Função	Chamada
`api.login(email, senha)`	`POST /auth/login`
`api.listarCategorias()`	`GET /categorias`
`api.listarChamados()`	`GET /chamados`
`api.buscarChamado(id)`	`GET /chamados/:id`
`api.criarChamado(dados)`	`POST /chamados`
`api.assumirChamado(id)`	`PATCH /chamados/:id/assumir`
`api.encerrarChamado(id)`	`PATCH /chamados/:id/encerrar`

Fonte: O autor (2026).

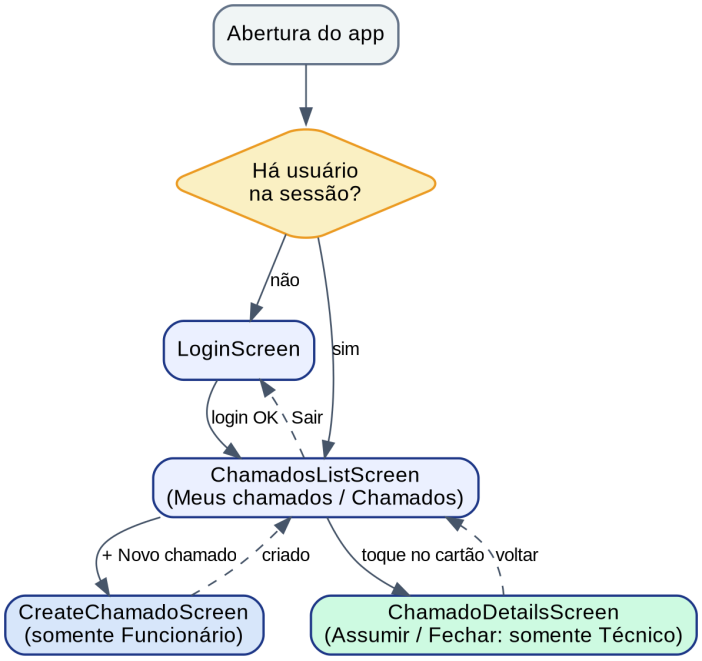
8 INTERFACE DO APLICATIVO

Esta seção apresenta as telas do aplicativo TicketHub em mockups de celular, organizadas por perfil. Cada tela traz sua finalidade, os elementos exibidos, as regras aplicadas e os casos de uso e requisitos correspondentes. A identidade visual usa azul `#1E3A8A` nos cabeçalhos e botões, fundo claro `#F5F5F5` e cores semânticas de status: âmbar para Aberto e verde para Em atendimento.

8.1 Navegação

Ao abrir o aplicativo, a sessão é restaurada do armazenamento local. Se houver usuário, a lista de chamados é exibida; caso contrário, a tela de login. A partir da lista, o funcionário acessa a criação de chamado e ambos os perfis acessam os detalhes de um chamado.

Figura 7 – Fluxo de navegação entre as telas



Fonte: O autor (2026), com base em AppNavigator.js.

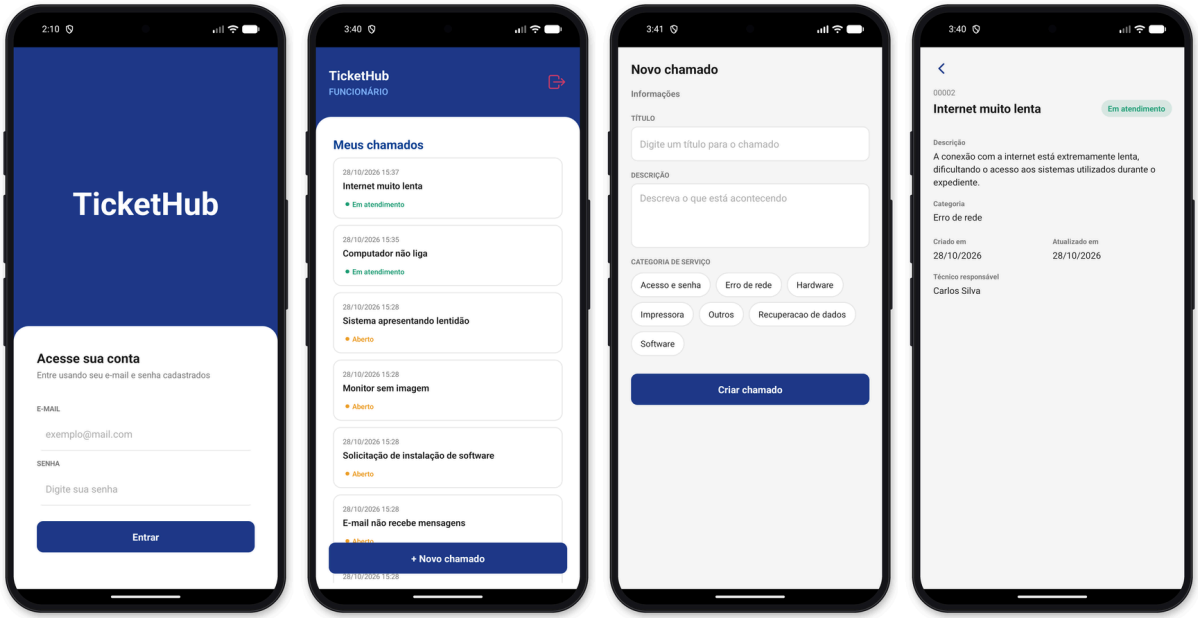
Tabela 32 – Telas do aplicativo

Tela	Perfil	Arquivo de captura	Casos de uso	Requisitos
Login	Ambos	`login.png`	UC01	RF01
Lista de chamados (funcionário)	Funcionário	`list-employee.png`	UC02, UC03	RF04, RF06, RF07

Tela	Perfil	Arquivo de captura	Casos de uso	Requisitos
Novo chamado	Funcionário	`create-employee.png`	UC05	RF05
Detalhes do chamado (funcionário)	Funcionário	`details-employee.png`	UC04	RF06, RF07, RF08
Lista de chamados (técnico)	Técnico	`list-tech.png`	UC02, UC03	RF04, RF06, RF07
Detalhes do chamado aberto (técnico)	Técnico	`details-tech.png`	UC04, UC06	RF08, RF09
Chamado assumido (técnico)	Técnico	`accept-ticket-tech.png`	UC04, UC07	RF10, RF11

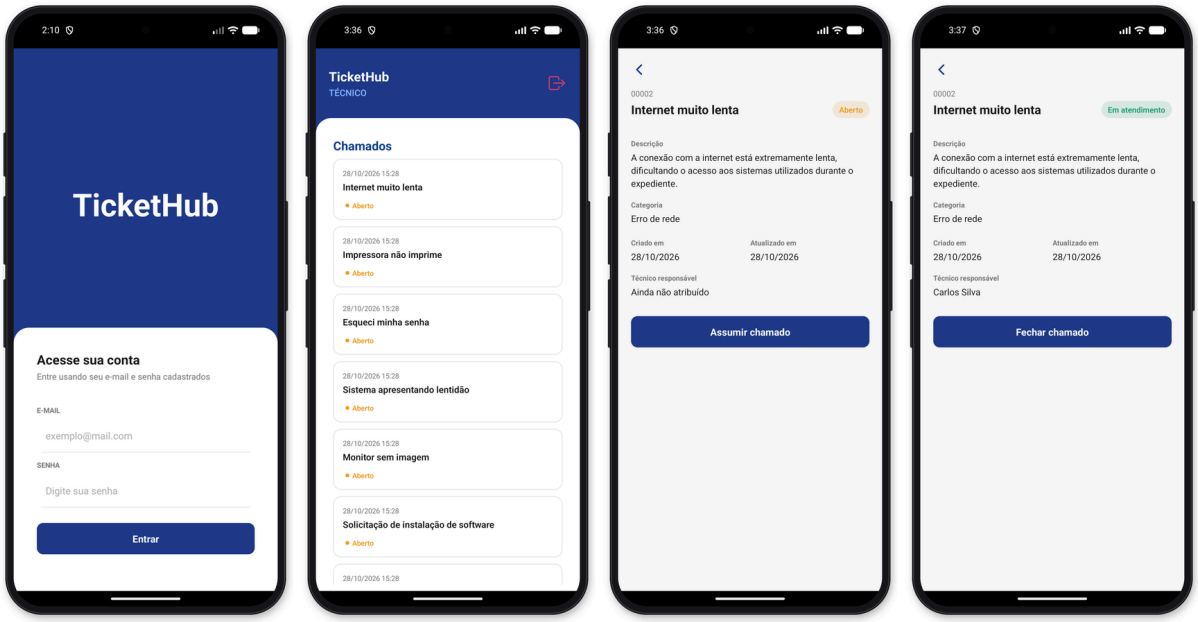
Fonte: O autor (2026).

Figura 8 – Visão geral do fluxo do funcionário: login, lista, novo chamado e detalhes



Fonte: O autor (2026).

Figura 9 – Visão geral do fluxo do técnico: login, fila, chamado aberto e chamado assumido

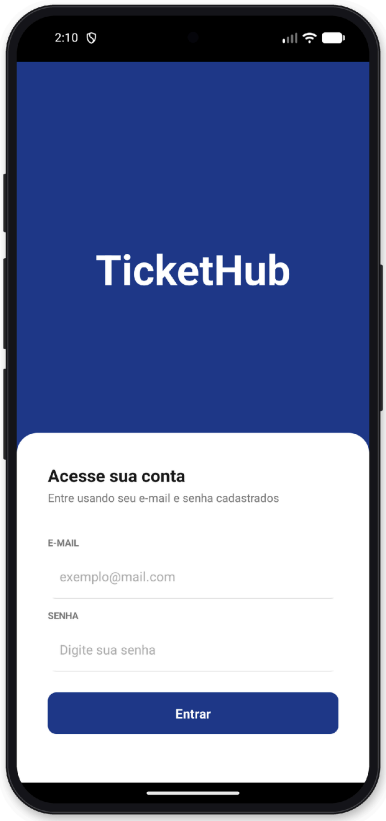


Fonte: O autor (2026).

8.2 Tela comum aos dois perfis

8.2.1 Login

Figura 10 – Tela de login



Finalidade

Autenticar o usuário com as credenciais fornecidas pela empresa. É a única tela exibida quando não há sessão ativa.

Elementos

Marca “TicketHub” sobre fundo azul.

Painel “Acesse sua conta” com a instrução “Entre usando seu e-mail e senha cadastrados”.

Campo E-MAIL (teclado de e-mail, sem capitalização automática).

Campo SENHA (texto oculto).

Botão “Entrar”, com indicador de carregamento durante a requisição.

Regras

Campos vazios: “Informe e-mail e senha.” (validado no app, sem chamar a API).

Credenciais incorretas: “Credenciais inválidas.” (mensagem única, RN03).

Não há link de cadastro nem de recuperação de senha (RN01).

Perfil e rastreio

Funcionário e técnico · UC01 · RF01, RN01 a RN03.

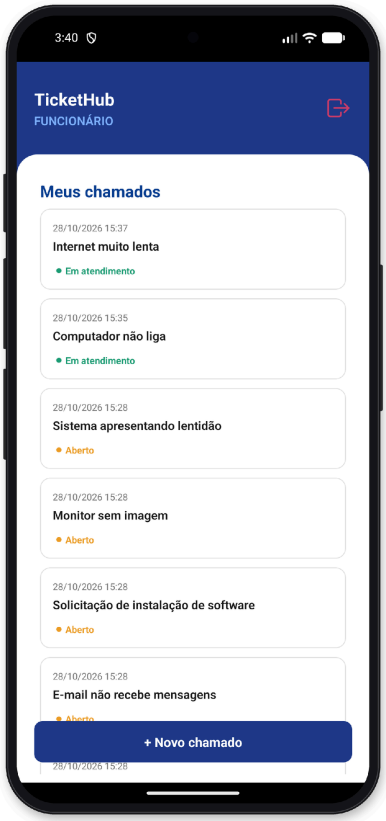
Fonte: Captura de tela do aplicativo TicketHub (2026).

8.3 Telas do funcionário

O funcionário (perfil `CLIENTE`) abre chamados e acompanha somente os seus. O cabeçalho da lista identifica o perfil como “FUNCIONÁRIO”.

8.3.1 Lista de chamados (Meus chamados)

Figura 11 – Lista “Meus chamados” do funcionário



Finalidade
Exibir os chamados abertos pelo próprio funcionário e permitir abrir um novo.

Elementos
Cabeçalho azul com “TicketHub”, o perfil “FUNCIONÁRIO” e o ícone de sair (UC02).

Título “Meus chamados”.

Cartões com data e hora da última atualização, título e status com ponto colorido (Aberto em âmbar, Em atendimento em verde).

Botão fixo “+ Novo chamado” no rodapé, exibido somente ao funcionário.

Comportamento
Ordenação do mais recentemente atualizado para o mais antigo: por isso o chamado “Internet muito lenta” (15:37), recém-assumido por um técnico, aparece no topo.

Atualiza ao puxar a tela para baixo e ao voltar de outra tela.

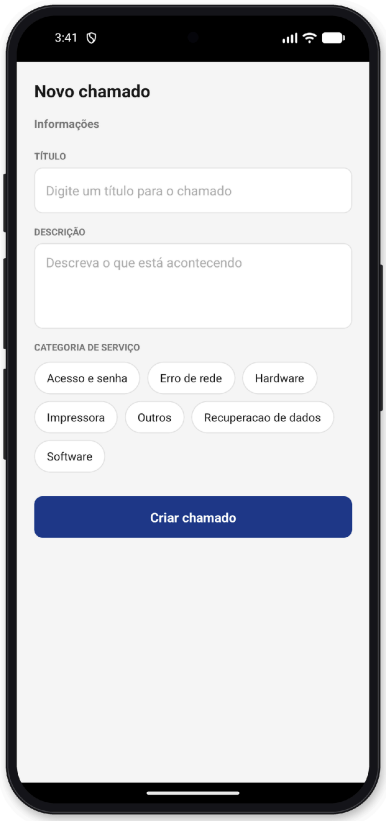
Lista vazia: “Nenhum chamado encontrado.”

Perfil e rastreio
Funcionário · UC02, UC03 · RF04, RF06, RF07, RN08.

Fonte: Captura de tela do aplicativo TicketHub (2026).

8.3.2 Novo chamado

Figura 12 – Tela de abertura de chamado



Finalidade
Registrar uma nova solicitação de suporte.

Elementos
Campo TÍTULO (“Digite um título para o chamado”).

Campo DESCRIÇÃO, multilinha (“Descreva o que está acontecendo”).

CATEGORIA DE SERVIÇO em chips selecionáveis: Acesso e senha, Erro de rede, Hardware, Impressora, Outros, Recuperacao de dados e Software.

Botão “Criar chamado”.

Regras
As categorias são carregadas da API (somente as ativas, em ordem alfabética); “Recuperacao de dados” aparece sem acento porque assim está cadastrada no seed do banco.

Todos os campos são obrigatórios: “Preencha título, descrição e categoria de atendimento.”

O chamado nasce com status Aberto e sem técnico (RN07). Ao criar, o app volta à lista.

Perfil e rastreio
Somente funcionário · UC05 · RF05, RN04 a RN07.

Fonte: Captura de tela do aplicativo TicketHub (2026).

8.3.3 Detalhes do chamado

Figura 13 – Detalhes do chamado na visão do funcionário



Finalidade
Permitir ao funcionário acompanhar o andamento de um chamado seu.

Elementos
Seta de voltar, número do chamado com cinco dígitos (00002), título e selo de status (“Em atendimento”).

Descrição e categoria (“Erro de rede”).

Datas “Criado em” e “Atualizado em”.

Técnico responsável (“Carlos Silva”); enquanto ninguém assumir, o campo mostra “Ainda não atribuído”.

Comportamento
Tela somente de consulta: o funcionário não possui botões de ação. A tela recarrega os dados sempre que ganha foco, refletindo ações do técnico.

Perfil e rastreio
Funcionário · UC04 · RF06, RF07, RF08, RN08.

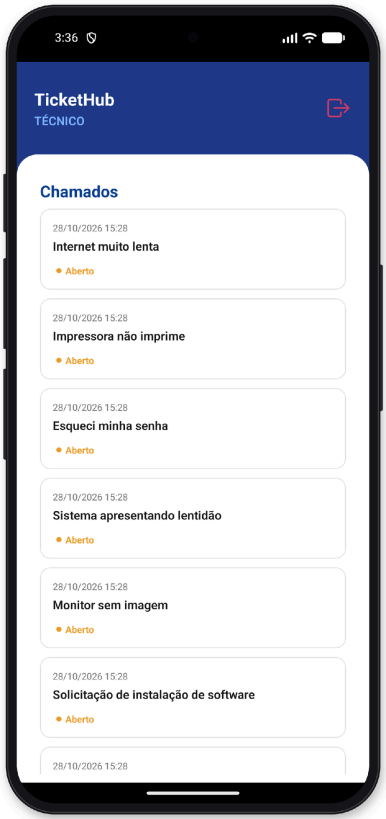
Fonte: Captura de tela do aplicativo TicketHub (2026).

8.4 Telas do técnico

O técnico (perfil `TECNICO`) visualiza a fila de chamados, assume chamados abertos e encerra aqueles pelos quais é responsável. O cabeçalho da lista identifica o perfil como “TÉCNICO”.

8.4.1 Lista de chamados (fila)

Figura 14 – Lista de chamados na visão do técnico



Finalidade
Mostrar a fila de chamados a atender.

Elementos
Cabeçalho azul com “TicketHub”, o perfil “TÉCNICO” e o ícone de sair.

Título “Chamados”.

Cartões com data e hora, título e status, no mesmo padrão da lista do funcionário.

Não há botão de novo chamado: o técnico não abre chamados (RN04).

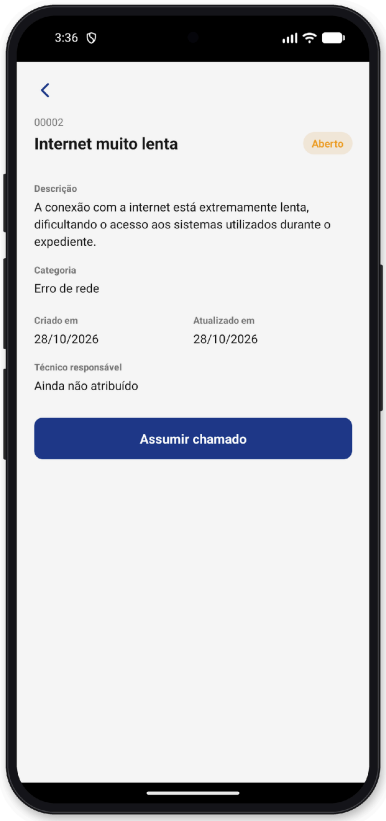
Comportamento
Exibe os chamados não encerrados (abertos e em atendimento), do mais antigo para o mais novo, de modo que a fila siga a ordem de chegada. Tocar em um cartão abre os detalhes.

Perfil e rastreio
Técnico · UC02, UC03 · RF04, RF06, RF07.

Fonte: Captura de tela do aplicativo TicketHub (2026).

8.4.2 Detalhes de chamado aberto (assumir)

Figura 15 – Detalhes de um chamado aberto na visão do técnico



Finalidade
Permitir ao técnico analisar o chamado e assumir o atendimento.

Elementos
Mesmos dados da tela de detalhes do funcionário.

Selo de status “Aberto” (âmbar).

Técnico responsável: “Ainda não atribuído”.

Botão “Assumir chamado”.

Regras
O botão aparece somente quando o usuário é técnico e o status é Aberto (RN10, RN11).

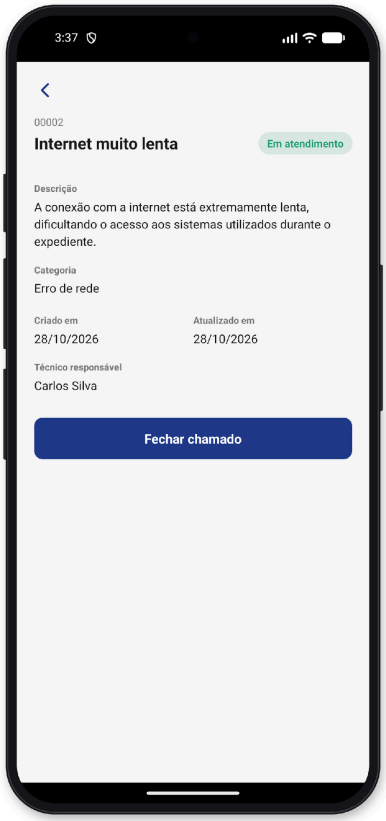
Ao tocar, o app chama `PATCH /chamados/:id/assumir`, o status passa a Em atendimento, o técnico logado vira o responsável e a tela é recarregada.

Perfil e rastreio
Técnico · UC04, UC06 · RF08, RF09, RN10 a RN12.

Fonte: Captura de tela do aplicativo TicketHub (2026).

8.4.3 Chamado assumido (fechar)

Figura 16 – Chamado assumido, com a opção de fechar



Finalidade
Concluir o atendimento depois de resolvido o problema.

Elementos
Selo de status “Em atendimento” (verde).
Técnico responsável: “Carlos Silva”.
Botão “Fechar chamado” no lugar de “Assumir chamado”.

Regras
O botão aparece somente quando o status é Em atendimento e o técnico logado é o responsável (RN13).
Ao tocar, o app chama `PATCH /chamados/:id/encerrar` e o status passa a Fechado; chamados já fechados não podem ser encerrados novamente (RN14).
A data “Atualizado em” é renovada pela trigger do banco (RN15).

Perfil e rastreio
Técnico responsável · UC07 · RF10, RF11, RN13, RN14.

Fonte: Captura de tela do aplicativo TicketHub (2026).

8.5 Cenário de ponta a ponta

O cenário abaixo, executado com o chamado nº 00002 (“Internet muito lenta”, categoria Erro de rede), mostra a integração entre os dois perfis e as capturas de tela correspondentes.

Tabela 33 – Cenário de validação com o chamado 00002

Passo	Perfil	Ação	Resultado observado	Captura
1	Técnico	Acessa a fila de chamados	Chamado “Internet muito lenta” com status Aberto (15:28)	`list-tech.png`
2	Técnico	Abre o chamado	Status Aberto; técnico “Ainda não atribuído”; botão “Assumir chamado”	`details-tech.png`
3	Técnico	Toca em “Assumir chamado”	Status Em atendimento; responsável Carlos Silva; surge “Fechar chamado”	`accept-ticket-tech.png`
4	Funcionário	Consulta “Meus chamados”	Chamado no topo, Em atendimento, atualizado às 15:37	`list-employee.png`
5	Funcionário	Abre o chamado	Detalhes com técnico responsável Carlos Silva e sem botões de ação	`details-employee.png`

Fonte: O autor (2026).

9 TESTES

9.1 Estratégia

Os testes automatizados são unitários e se concentram nos services (`authService` e `chamadoService`), que reúnem toda a regra de negócio. Os repositories são substituídos por mocks do Jest, portanto os testes não exigem banco de dados nem servidor em execução, são rápidos e determinísticos, e cada um verifica uma regra específica, inclusive o código HTTP esperado. Isso só é possível graças à injeção de dependência por factory (seção 5.3).

```
cd backend
npm install
npm test                  # execução única (jest --runInBand)
npx jest --coverage       # relatório de cobertura
```

9.2 Inventário dos testes automatizados

Tabela 34 – Testes de authService (4)

Teste	Regra verificada
Autentica com credenciais válidas e retorna token	Devolve token e dados do usuário; `senha_hash` nunca é retornado (RN17)
Rejeita e-mail inexistente	401 (RN03)
Rejeita senha incorreta	401 (RN03)
Rejeita quando e-mail ou senha não são informados	400, sem consultar o banco

Fonte: O autor (2026).

Tabela 35 – Testes de chamadoService (14)

Grupo	Teste	Regra verificada
criar	Cliente abre chamado com dados válidos	Cria com `clienteId` do usuário; status ABERTO
criar	Técnico não pode abrir chamado	403 e nada é gravado (RN04)
criar	Rejeita chamado sem título	400 (RN05)
criar	Rejeita categoria inexistente ou inativa	400 (RN06)
listar	Cliente vê apenas os próprios chamados	Usa `listarPorCliente` (RN08)
listar	Técnico vê a fila de chamados	Usa `listarParaTecnico`
buscarDetalhes	Cliente não vê chamado de outro cliente	403 (RN08)
buscarDetalhes	Lança 404 quando o chamado não existe	404

Grupo	Teste	Regra verificada
assumir	Técnico assume um chamado aberto	Status EM_ATENDIMENTO (RN10)
assumir	Não permite assumir chamado em atendimento	409 (RN11)
assumir	Cliente não pode assumir chamado	403 (RN10)
encerrar	Técnico responsável encerra o chamado	Status FECHADO
encerrar	Técnico que não é o responsável não encerra	403 (RN13)
encerrar	Não permite encerrar chamado já fechado	409 (RN14)

Fonte: O autor (2026).

9.3 Teste manual de ponta a ponta

Além dos testes unitários, o fluxo completo foi validado manualmente no aplicativo, com a API e o banco em execução, conforme o cenário da seção 8.5. O roteiro rápido de validação é:

- Entrar como funcionário, tocar em “+ Novo chamado” e criar um chamado.
- Sair e entrar como técnico.
- Abrir o chamado, tocar em “Assumir chamado” e depois em “Fechar chamado”.
- Voltar como funcionário e conferir o status “Fechado”.

9.4 Lacunas conhecidas

Tabela 36 – O que não está coberto por testes

Área	Situação	Sugestão
Middlewares `auth` **e** `authorize`	Sem testes	Testes unitários com `req`, `res` e `next` falsos
Controllers e rotas (HTTP)	Sem testes	Testes de integração com `supertest`
Repositories e trigger	Sem testes	Testes de integração com PostgreSQL de teste
Aplicativo (interface)	Sem testes automatizados	`jest-expo` e React Native Testing Library

Fonte: O autor (2026).

10 SEGURANÇA E LGPD

Esta seção resume as medidas de segurança, as lacunas identificadas e o plano de adequação à Lei Geral de Proteção de Dados Pessoais (Lei nº 13.709/2018). O conteúdo tem finalidade técnica e acadêmica e não constitui parecer jurídico.

10.1 Dados pessoais tratados

Tabela 37 – Dados pessoais tratados pelo sistema

Dado	Onde	Observação
Nome e e-mail	`usuario.nome`, `usuario.email`	Dados pessoais; o e-mail é o identificador de login
Hash da senha	`usuario.senha_hash`	Credencial protegida por bcrypt
Perfil e situação da conta	`usuario.tipo`, `usuario.ativo`	Dado associado ao titular
Título e descrição do chamado	`chamado.titulo`, `chamado.descricao`	Texto livre: pode conter dados pessoais digitados pelo usuário
Vínculos cliente e técnico	`chamado.cliente_id`, `tecnico_id`	Registro de quem atendeu quem

Fonte: O autor (2026).

10.2 Medidas implementadas

Tabela 38 – Medidas de segurança implementadas

Medida	Implementação
Senhas com hash bcrypt	A senha nunca é armazenada em texto claro (`authService.js`)
Hash nunca exposto	O login retorna apenas id, nome, e-mail e tipo
Mensagem de login genérica	“Credenciais inválidas.” evita a enumeração de contas
Contas inativas bloqueadas	`WHERE ativo = TRUE` na busca de usuário
Autenticação por JWT	Token assinado, com expiração configurável
Controle de acesso por perfil	`authorize(...)` nas rotas sensíveis
Isolamento de dados do cliente	O cliente só lista e vê os próprios chamados
Defesa em profundidade	Perfil validado na rota e novamente no service
SQL parametrizado	`$1, $2...` em todas as consultas (mitiga SQL Injection)
Validação de entrada	Título, descrição e categoria validados antes de gravar
Sem autocadastro	Contas criadas somente pelo T.I.
Segredos fora do código	`.env` ignorado pelo Git; apenas `.env.example` versionado

Fonte: O autor (2026).

10.3 Lacunas e plano de adequação

Tabela 39 – Lacunas identificadas e correções propostas

Prioridade	Lacuna	Correção proposta
Alta	Técnico vê chamados em atendimento por colegas (consulta retorna todos os não fechados)	Filtrar por `status = ABERTO` ou `tecnico_id` do próprio técnico
Alta	Tráfego sem criptografia (`http://`)	HTTPS com proxy reverso e TLS em produção
Alta	Token guardado em AsyncStorage (sem criptografia)	`expo-secure-store`
Média	Sem limite de tentativas de login	`express-rate-limit` na rota de login
Média	Mensagens internas de erro expostas em erros 500	Handler global de erros com mensagem genérica
Média	CORS aberto	Restringir as origens permitidas
Média	Sem política de privacidade nem fluxo de direitos do titular	Política no app, aviso no campo de descrição e canal do titular
Média	Sem política de retenção e eliminação de dados	Prazos por finalidade e rotina de anonimização
Baixa	Sem auditoria, sem logout no servidor e sem `helmet`	Logs de segurança, lista de revogação e cabeçalhos de segurança

Fonte: O autor (2026).

Essas lacunas não significam que todos os princípios da LGPD estejam violados; indicam pontos a avaliar e aprimorar antes de qualquer uso em produção. No plano de governança, destacam-se: mapear dados e finalidades, definir as bases legais, publicar política de privacidade, orientar o usuário a não inserir dados pessoais desnecessários na descrição, definir prazos de retenção, preparar resposta a incidentes e avaliar a necessidade de encarregado (DPO).

10.4 Boas práticas de operação

- Nunca versionar o `.env` e rotacionar o `JWT_SECRET` se for exposto.
- Trocar as senhas de teste e remover os usuários do seed fora do desenvolvimento.
- Usar usuário de banco com menor privilégio e backups criptografados.
- Manter dependências atualizadas (`npm audit`).
- Não registrar senhas, tokens ou descrições de chamados em logs.

11 LIMITAÇÕES E TRABALHOS FUTUROS

11.1 Limitações conhecidas

Por ser um MVP, o sistema possui limitações assumidas conscientemente. As principais estão listadas a seguir; as de segurança estão detalhadas na seção 10.

Tabela 40 – Limitações do MVP

Área	Limitação	Impacto
Backend	A fila do técnico retorna todos os chamados não fechados, inclusive os em atendimento por outros técnicos	Visibilidade além da regra de negócio
Backend	Assumir não é atômico: dois técnicos simultâneos podem assumir o mesmo chamado (o último vence)	Inconsistência de responsável
Backend	`id` de rota e `categoriaId` não são validados quanto ao tipo; tamanho do título só é limitado pelo banco	Erro 500 com detalhe interno
Backend	Sem paginação, sem refresh token, sem migrations (o `schema.sql` recria tudo)	Escalabilidade e evolução do banco
Aplicativo	Token expirado não desloga automaticamente e a sessão é restaurada sem validar o token no servidor	Erros após o prazo de 8 h
Aplicativo	Erros ao carregar a lista vão só para o console	Falha silenciosa
Aplicativo	Sem confirmação antes de assumir ou fechar um chamado	Cliques acidentais
Aplicativo	URL da API fixa no código; sem tela de recuperação de senha	Troca manual por ambiente; depende do T.I.
Produto	Sem comentários, anexos, notificações, prioridade, SLA, relatórios, reabertura e perfil administrador	Escopo reduzido do MVP

Fonte: O autor (2026).

11.2 Roadmap sugerido

Tabela 41 – Evolução planejada

Fase	Itens
1 – Correções e robustez	Corrigir a fila do técnico; tornar a atribuição atômica (`UPDATE ... WHERE status = 'ABERTO'`); validar entradas; handler global de erros; logout automático no 401; exibir erros de carregamento.
2 – Segurança e LGPD	HTTPS, `helmet`, CORS restrito e rate limiting; `expo-secure-store`; política de privacidade; rotinas de retenção e anonimização.
3 – Qualidade	Testes de middlewares e de integração; testes de componentes; migrations; CI com GitHub Actions; configuração por ambiente.
4 – Funcionalidades	Comentários e histórico; anexos; notificações push; prioridade e SLA; filtros, busca e paginação; perfil administrador; reabertura de chamado.

Fonte: O autor (2026).

12 CONSIDERAÇÕES FINAIS

O TicketHub atingiu o objetivo proposto para o MVP: oferecer um fluxo simples e rastreável de atendimento de chamados (abrir, assumir e encerrar), com dois perfis de acesso, status e responsável sempre visíveis e regras de negócio protegidas no backend. A arquitetura em camadas com injeção de dependência permitiu testar as regras sem infraestrutura, e o aplicativo mobile entrega a experiência completa para funcionário e técnico.

As limitações e o roadmap registrados neste documento mostram os caminhos de evolução, com prioridade para a correção da fila do técnico, o reforço de segurança (HTTPS, armazenamento seguro do token e limitação de tentativas) e a adequação à LGPD, etapas necessárias antes de qualquer uso em produção.

APÊNDICE A – INSTALAÇÃO E EXECUÇÃO

A.1 Pré-requisitos

- Node.js 18 ou superior e npm.
- PostgreSQL 13 ou superior.
- Git.
- Aplicativo Expo Go no celular (Android ou iOS) ou um emulador.

A.2 Passo a passo

1. Banco de dados: criar o banco e executar o script na raiz do repositório.

```
createdb -U postgres tickethub
psql -U postgres -d tickethub -f schema.sql
```

2. Backend: instalar dependências, configurar o `.env` (a partir de `.env.example`) e iniciar a API. A verificação é feita em `http://localhost:3000/health`, que deve responder `{"status":"ok"}`.

```
cd backend
npm install
cp .env.example .env
npm run dev
```

Tabela 42 – Variáveis de ambiente do backend

Variável	Descrição
`PORT`	Porta da API (padrão 3000)
`DATABASE_URL`	String de conexão com o PostgreSQL
`JWT_SECRET`	Segredo longo e aleatório para assinar os tokens
`JWT_EXPIRES_IN`	Validade do token (padrão 8h)

Fonte: O autor (2026).

3. Aplicativo: instalar dependências, ajustar a `BASE_URL` em `frontend/src/services/api.js` e iniciar o Expo.

```
cd frontend
npm install
npx expo start
```

Tabela 43 – Valor de BASE_URL por ambiente

Onde o app roda	BASE_URL
Emulador iOS	`http://localhost:3000/api`
Emulador Android	`http://10.0.2.2:3000/api`

Onde o app roda	BASE_URL
Celular físico (Expo Go)	`http://<IP-DA-MÁQUINA>:3000/api` (mesma rede Wi-Fi)

Fonte: O autor (2026).

A.3 Credenciais de desenvolvimento

Tabela 44 – Contas de teste (somente desenvolvimento)

Perfil	E-mail	Senha
Funcionário (cliente)	antonio@test.com	123456
Técnico	carlos.silva@test.com	123456

Fonte: O autor (2026).